```
    "You have no interesting cards,"
  end
end

puts pick_a_card([
  {rank: 7, suit: "Hearts"},
  {rank: 8, suit: "Diamonds"},
  {rank: 9, suit: "Clubs"}
])
```

*produces:*

```
You have three consecutive cards
```

In this example, the pattern matches if the first card has a rank, the second card has a rank that is `rank + 1`, and the third card has `rank + 2`.

Pinning also works on single-line patterns, it's not limited to patterns within case statements.

## Guard Clauses

There's one other thing that you can do with patterns. You can use a boolean statement to add a guard clause at the end of the pattern, the pattern only matches if the clause is true.

```
def pick_a_card(cards)
  cards = cards.sort_by { _1[:rank] }
  case cards
  in [{rank:}, {rank: ^(rank + 1)}, {rank: ^(rank+ 2)}] if rank > 6
    "You have three consecutive high cards"
  else
    "You have no interesting cards,"
  end
end

puts pick_a_card([
  {rank: 7, suit: "Hearts"},
  {rank: 8, suit: "Diamonds"},
  {rank: 9, suit: "Clubs"}
])
```

*produces:*

```
You have three consecutive high cards
```

In the above example, the clause `if rank > 6` limits the pattern to match only cases where the lowest rank of the three cards is greater than 6. As you can see, variables assigned as part of the pattern can be used in the clause. You can use `unless` instead of `if` here, in which case the pattern matches when the `unless` clause is false.

## Custom Pattern Matching

Our "pick a card" examples so far have had our card data stored in a Ruby hash, but it's not unlikely that we'd rather have them stored in their own class.

At this point, we can no longer pattern match against our card class because a hash match no longer works:

```
class Card
  attr_accessor :rank, :suit
```